AMARA

Backend API Specification & Integration Guide

Complete Production-Grade Contracts, Response Schemas, Database DDL & SMTP Configuration

Document Version:	3.2 (Production Release)
Date Generated:	October 28, 2026
Target Audience:	Backend Developers, API Engineers, DBAs, QA
Client Application:	AMARA React Native Client (iOS & Android)
Primary Identity:	Email Authentication (Gmail SMTP Verified)
Test Accounts:	rahul@amara.com / ananya.roy@amara.com (Password@123)
Universal Test OTP:	123456

AMARA Dating Application - Complete Backend API Specification & Response Schemas

This document contains the complete, production-grade specification for all endpoints used by the AMARA React Native application. Every endpoint includes the HTTP method, endpoint route, required headers, query parameters, request body schemas, exact expected response JSONs (Success & Error), and data types.

Document Overview: What is Included in this Specification

This specification covers all **13 Functional Modules** and **38 Endpoints** required by the AMARA mobile frontend. Every single endpoint includes its HTTP method, URL path, required headers, query parameters, request payload schema, and **exact full JSON responses (Success 200/201 and Error codes)** directly embedded in this single document for easy reference and implementation by backend developers.

Functional Modules Index

1. **Authentication & Account Lifecycle**: Email registration, OTP email verification, user login, password reset, account deactivation & deletion.
2. **Onboarding Flow**: Multi-step profile setup, gender/orientation selection, AI/manual selfie verification, initial photo upload.
3. **User Profile Management**: Fetching profile (`/profile/me` & `/profile/get`), updating bio, details, preferences, and contact info.
4. **Media & Photo Gallery**: Uploading up to 6 gallery photos, setting primary photo, deletion, and ordering.
5. **Discovery & Feed**: Home feed matching, online users feed, recent profiles, and deep search filters.
6. **Connections & Match Requests**: Sending connect requests, managing sent/received inboxes, accepting/rejecting requests, connection status checks.
7. **Telegram Deep-Link Integration**: Linking Telegram username, fetching chat deep links (`https://t.me/<username>`), unlinking.
8. **Subscription & Razorpay Payments**: Creating Razorpay orders, webhook/server payment verification, active plan status, remaining days.
9. **Geolocation & Location Discovery**: Updating GPS coordinates, location history, and discovering nearby profiles.
0. **Help Desk & Support**: Submitting support tickets, listing ticket history, closing tickets.
1. **Privacy & Legal Compliance**: Terms & privacy acceptance tracking and verification.
2. **In-App Notifications**: Real-time notifications for connection requests, subscriptions, and read state management.
3. **Presence, Moderation & Safety**: Real-time online/offline toggling, reporting abusive users, blocking profiles.
4. **Transactional & Notification Email Templates**: Production-ready responsive HTML & plain-text email templates for OTP verification, password reset, welcome onboarding, connection requests, match approvals, and subscription activation.

Master Endpoints Catalog & Quick Jump

#	Functional Domain	Endpoint Title	Method	Path	Auth?	Success Code	Specification Link
1	Auth	Send OTP (Email)	POST	`/auth/send-otp`	No	200 OK	Section 1.1
2	Auth	Initiate Email Registration	POST	`/register`	No	200 OK	Section 1.2

3	Auth	Verify Register OTP (Email)	POST	`/verify-register/otp`	No	200 OK	Section 1.3
4	Auth	User Login (Email + Password)	POST	`/login`	No	200 OK	Section 1.4
5	Auth	Forgot Password - Send OTP (Email)	POST	`/auth/forgot-password/send-otp`	No	200 OK	Section 1.5
6	Auth	Forgot Password - Reset (Email)	POST	`/auth/forgot-password/reset`	No	200 OK	Section 1.6
7	Auth	Deactivate / Activate Account	POST / PUT	`/account/deactivate`, `/account/activate`	Bearer	200 OK	Section 1.7
8	Auth	Permanent Account Deletion	DELETE	`/account/delete`	Bearer	200 OK	Section 1.8
9	Onboarding	Gender & Orientation	POST	`/profile/gender-orientation`	Bearer	200 OK	Section 2.1
10	Onboarding	Biometric Selfie Verification	PUT	`/profile/selfie/verify/{userId}`	Bearer	200 OK	Section 2.2
11	Onboarding	Upload Gallery Image	POST	`/profile/upload-image`	Bearer	200 OK	Section 2.3
12	Onboarding	Complete Profile Setup	POST	`/profile/{userId}/setup`	Bearer	200 OK	Section 2.4
13	Onboarding	Profile Completion %	GET	`/profile/completion`	Bearer	200 OK	Section 2.5
14	Profile	Fetch My Profile	GET	`/profile/me`	Bearer	200 OK	Section 3.1
15	Profile	Fetch Target User Profile	GET	`/profile/get?userId={id}`	Bearer	200 OK	Section 3.1
16	Profile	Update Full Profile	PUT	`/profile/update`	Bearer	200 OK	Section 3.2
17	Profile	Update Basic Info	PUT	`/profile/basic`	Bearer	200 OK	Section 3.3
18	Profile	Update Personal Details	PUT	`/profile/personal`	Bearer	200 OK	Section 3.4
19	Profile	Update Search Preferences	PUT	`/profile/preferences`	Bearer	200 OK	Section 3.5
20	Profile	Gallery Photos Management	GET / DELETE	`/images/users/{userId}`, `/images/{id}`	Bearer	200 OK	Section 3.6
21	Discovery	Online Feed	GET	`/dashboard/online`	Bearer	200 OK	Section 4.1
22	Discovery	Recent Profiles Feed	GET	`/dashboard/recent`	Bearer	200 OK	Section 4.2
23	Discovery	Advanced Search & Filter	POST	`/users/filter`	Bearer	200 OK	Section 4.3

24	Connections	Send Connection Request	POST	`/connections/send`	Bearer	200 OK	Section 5.1
25	Connections	Get Sent Requests	GET	`/connections/sent?userId={id}`	Bearer	200 OK	Section 5.2
26	Connections	Get Received Requests	GET	`/connections/received?userId={id}`	Bearer	200 OK	Section 5.3
27	Connections	Accept Connection Request	PUT	`/connections/accept`	Bearer	200 OK	Section 5.4
28	Connections	Check Connection Status	GET	`/connections/status`	Bearer	200 OK	Section 5.5
29	Telegram	Connect Telegram Username	POST	`/telegram/connect`	Bearer	200 OK	Section 6.1
30	Telegram	Fetch Direct Telegram Link	GET	`/telegram/link?userId={id}`	Bearer	200 OK	Section 6.2
31	Telegram	Disconnect Telegram	DELETE	`/telegram/disconnect`	Bearer	200 OK	Section 6.3
32	Subscription	Create Razorpay Order	POST	`/razorpay/create-order`	Bearer	200 OK	Section 7.1
33	Subscription	Verify Payment & Activate	POST	`/razorpay/verify`	Bearer	200 OK	Section 7.2
34	Subscription	Subscription Status & Days	GET	`/subscriber/status`	Bearer	200 OK	Section 7.3
35	Location	Add User Location	POST	`/location/add`	Bearer	200 OK	Section 8.1
36	Location	Get Location History	GET	`/location/history/{userId}`	Bearer	200 OK	Section 8.2
37	Location	Get Nearby Profiles	GET	`/location/nearby`	Bearer	200 OK	Section 8.3
38	Support	Create Support Ticket	POST	`/support/create`	Bearer	200 OK	Section 9.1
39	Support	Get My Tickets	GET	`/support/my`	Bearer	200 OK	Section 9.2
40	Support	Close Support Ticket	PUT	`/support/close/{ticketId}`	Bearer	200 OK	Section 9.3
41	Legal & Safety	Accept Privacy Policy	POST	`/privacy/accept`	Bearer	200 OK	Section 10.1
42	Legal & Safety	Check Privacy Status	GET	`/privacy/status`	Bearer	200 OK	Section 10.2
43	Legal & Safety	Report a Profile	POST	`/reports/report`	Bearer	200 OK	Section 10.3
44	Legal & Safety	Update Online Status	PUT	`/status/online`	Bearer	200 OK	Section 10.4
45	Legal & Safety	In-App Notifications	GET	`/notification`	Bearer	200 OK	Section 10.5

46	Settings & Alerts	Get Notification Preferences	GET	`/notification/preferences`	Bearer	200 OK	Section 10.6
47	Settings & Alerts	Update Notification Preferences	PUT	`/notification/preferences`	Bearer	200 OK	Section 10.6
48	Email Service	6 Full Production Email Templates	N/A	Nodemailer / SES / SendGrid	System	Ready	Section 11

Global API Standards

- **Base URL**: `https://api.amara.com/api/v1` (or local environment `http://localhost:8080/api/v1`)
- **Headers**:
- `Accept: application/json`
- `Content-Type: application/json` (unless file upload: `multipart/form-data`)
- `Authorization: Bearer <JWT_TOKEN>` (for all authenticated endpoints)
- **Standard Error Response Format**:

```
{
  "status": "error",
  "message": "Human-readable explanation of error.",
  "code": "ERROR_CODE_STRING"
}
```

---

1. Authentication & Account Lifecycle

OTP Delivery & Backend SMTP Architecture

Requirement / Component	Needed for OTPs?	Notes
Backend SMTP Configuration	✅ **Yes**	The backend simply needs standard Gmail SMTP credentials to send the verification email.
Push Notifications (FCM / APNs)	❌ **No**	Push notifications are not used for sending OTPs.
In-App Notification Preferences	❌ **No**	User notification settings only apply to date and match alerts. OTPs are mandatory security verifications and must always be delivered.
Client Auto-Read / SMS Retriever	❌ **No**	Mobile operating systems sandbox email; users enter the 6-digit code manually or via clipboard paste.

Backend SMTP Configuration Details (Gmail SMTP)

To enable real email dispatch for registration OTPs (`POST /register`), password reset codes (`POST /forgot-password/send-otp`), and notification digests:

- **Host**: `smtp.gmail.com`
- **Port**: `587` (STARTTLS) or `465` (SSL)
- **Auth**: Standard Google App Password (generated via Google Account > Security > 2-Step Verification > App passwords)
- **Environment Variables**:

```
SMTP_HOST=smtp.gmail.com
SMTP_PORT=587
SMTP_SECURE=false
SMTP_USER=your-amara-service-account@gmail.com
SMTP_PASS=your-16-char-google-app-password
SMTP_FROM="AMARA Dating <no-reply@amara-app.com>"
```

Pre-Configured Test Credentials & QA Accounts

For local development, API integration testing, and mobile client verification:

Account Type	Flow / Purpose	Email Address	Password	Static Test OTP
Male User	Discovery Feed, Browse Profiles & Send Invitations	`rahul@amara.com` *(or `sailakshmiborra45@gmail.com`)*	`Password@123`	**`123456`**
Female User	Invitations Inbox, Approve & Decline Dates	`ananya.roy@amara.com`	`Password@123`	**`123456`**
Male User (Alt)	Secondary Discovery User	`arjun.kapoor@amara.com`	`Password@123`	**`123456`**
Female User (Alt)	Secondary Inbox User	`priya.sen@amara.com`	`Password@123`	**`123456`**

> **Note on Static OTP**: When verifying OTP for registration, login, or password reset, the development environment always accepts **`123456`**.

1.1 Send OTP (Email)

Sends a 6-digit OTP code to the user's email address for verification.

- **Endpoint**: `POST /auth/send-otp`
- **Auth Required**: No
- **Request Body**:

```
{
  "email": "arjun@example.com"
}
```

- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "OTP sent successfully to arjun@example.com",
  "sessionId": "sess_1712345678_ab12c",
  "data": {
    "sessionId": "sess_1712345678_ab12c",
    "email": "arjun@example.com"
  }
}
```

- **Error Response (400 Bad Request)**:

```
{
  "status": "error",
  "message": "Please enter a valid email address.",
  "code": "INVALID_IDENTIFIER"
}
```

1.2 Initiate Registration

Starts user account registration using **email** as the primary identity, generates a verification session, and sends the 6-digit OTP code to the registered email address.

- **Endpoint**: `POST /register`
- **Auth Required**: No
- **Request Body**:

```
{
  "name": "Arjun Sharma",
  "email": "arjun@example.com",
  "password": "Password@123",
  "confirmPassword": "Password@123"
}
```

- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Registration initiated. Verification code sent to arjun@example.com.",
  "sessionId": "sess_reg_arjun_123",
  "data": {
    "sessionId": "sess_reg_arjun_123",
    "email": "arjun@example.com",
    "name": "Arjun Sharma"
  }
}
```

- **Error Response (400 Bad Request / 409 Conflict)**:

```
{
  "status": "error",
  "message": "An account with this email address already exists. Please log in.",
  "code": "DUPLICATE_ENTRY"
}
```

1.3 Verify Registration OTP

Validates the OTP sent to the user's email and returns the JWT authentication token, user ID, and profile.

- **Endpoint**: `POST /verify-register/otp`
- **Auth Required**: No
- **Request Body**:

```
{
  "email": "arjun@example.com",
  "otp": "123456",
  "sessionId": "sess_reg_arjun_123"
}
```

- **Success Response (200 OK)**:


```
{
  "status": "success",
  "message": "User registered successfully",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiJ1c3JfbWFuXzEwNiIsImlhdCI6MTcxMjM0NTY3OH0.mock_signature",
  "userId": "usr_man_106",
  "user": {
    "id": 106,
    "userId": "usr_man_106",
    "name": "Arjun Sharma",
    "displayName": "Arjun Sharma",
    "email": "arjun@example.com",
    "mobile": "",
    "gender": "man",
    "telegramUsername": "",
    "profile": {
      "userId": "usr_man_106",
      "name": "Arjun Sharma",
      "displayName": "Arjun Sharma",
      "age": 24,
      "gender": "man",
      "photos": [],
      "verifiedSelfie": false
    }
  }
}
```


- **Error Response (400 Bad Request)**:


```
{
  "status": "error",
  "message": "Invalid OTP. Please check and try again.",
  "code": "INVALID_OTP"
}
```

1.4 User Login

Authenticates an existing user via email address and password.

- **Endpoint**: `POST /login`
- **Auth Required**: No
- **Request Body**:


```
{
  "email": "arjun@example.com",
  "password": "Password@123"
}
```


- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Login successful",
  "token": "eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9...",
  "userId": "usr_man_106",
  "user": {
    "id": 106,
    "userId": "usr_man_106",
    "name": "Arjun Sharma",
    "displayName": "Arjun",
    "email": "arjun@example.com",
    "mobile": "",
    "gender": "man",
    "telegramUsername": "arjun_telegram"
  }
}
```


- **Error Response (404 Not Found)**:


```
{
  "status": "error",
  "message": "Account not found. Please register first.",
  "code": "ACCOUNT_NOT_FOUND"
}
```


- **Error Response (401 Unauthorized)**:


```
{
  "status": "error",
  "message": "Invalid credentials. Please verify your password.",
  "code": "INVALID_CREDENTIALS"
}
```

1.5 Forgot Password - Send OTP

Sends a 6-digit password reset verification code to the user's registered email address.

- **Endpoint**: `POST /forgot-password/send-otp`
- **Auth Required**: No
- **Request Body**:


```
{
  "email": "arjun@example.com"
}
```


- **Success Response (200 OK)**:


```
{
  "status": "success",
  "message": "Password reset code sent to your registered email address.",
  "sessionId": "sess_forgot_1712345",
  "data": {
    "sessionId": "sess_forgot_1712345",
    "email": "arjun@example.com"
  }
}
```

1.6 Forgot Password - Reset

Verifies the OTP sent to the email and sets the new password.

- **Endpoint**: `POST /forgot-password/reset`
- **Auth Required**: No
- **Request Body**:

```
{
  "email": "arjun@example.com",
  "otp": "123456",
  "newPassword": "NewPassword@123"
}
```

- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Password reset successfully. You can now login with your new password."
}
```

1.7 Deactivate / Activate Account

- **Endpoint (Deactivate)**: `POST /account/deactivate?userId=usr_man_106`
- **Endpoint (Activate)**: `PUT /account/activate?userId=usr_man_106`
- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Account status updated successfully"
}
```

1.8 Permanent Account Deletion

- **Endpoint**: `DELETE /account/{userId}`
- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Account deleted successfully and all associated personal data purged."
}
```

---

2. Onboarding & Profile Setup

2.1 Set Gender & Sexual Orientation (Step 1)

- **Endpoint**: `POST /profile/gender-orientation`
- **Headers**: `Authorization: Bearer <token>`
- **Request Body**:

```
{
  "userId": "usr_man_106",
  "gender": "man",
  "orientation": "straight"
}
```

(Options for gender: `man`, `woman`, `lgbtqia`. Options for orientation: `straight`, `lgbtqia`)

- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Gender & orientation saved successfully",
  "data": {
    "gender": "man",
    "orientation": "straight"
  }
}
```

2.2 Biometric Selfie Verification (Step 3)

- **Endpoint**: `PUT /profile/selfie/verify/{userId}`
- **Headers**: `Authorization: Bearer <token>`
- **Request Body**: Empty or optional `{}`
- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Selfie biometric verification completed successfully",
  "data": {
    "userId": "usr_man_106",
    "photoVerified": true,
    "selfieVerified": true,
    "verifiedSelfie": true
  }
}
```

2.3 Upload Gallery Image (Step 4 & Edit Profile)

Uploads an image file to the user's gallery.

- **Endpoint**: `POST /profile/upload-image`
- **Headers**:
- `Authorization: Bearer <token>`
- `Content-Type: multipart/form-data`
- **Form Data**:
- `image` or `photo`: (Binary image file, JPG/PNG)
- `userId`: `"usr_man_106"`
- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Image uploaded successfully",
  "id": 14,
  "imageUrl": "https://storage.amara.com/users/usr_man_106/gallery_14.jpg",
  "data": {
    "id": 14,
    "userId": "usr_man_106",
    "imageUrl": "https://storage.amara.com/users/usr_man_106/gallery_14.jpg",
    "isPrimary": false,
    "createdAt": "2026-10-13T16:00:00.000Z"
  }
}
```

2.4 Profile Setup Completion (Steps 6 to 8)

Submits initial comprehensive profile details including name, bio, physical characteristics, habits, and primary photo.

- **Endpoint**: `POST /profile/{userId}/setup`
- **Headers**: `Authorization: Bearer <token>`
- **Request**: Can accept multipart form data (with `photo` binary file) OR JSON body:

```
{
  "name": "Arjun Sharma",
  "displayName": "Arjun",
  "gender": "man",
  "orientation": "straight",
  "age": 24,
  "dob": "2002-05-14",
  "bio": "Software engineer in Bangalore. Coffee lover, weekend hiker.",
  "language": "English, Hindi, Telugu",
  "appearance": "Athletic",
  "bodyType": "Fit",
  "height": 178,
  "englishLevel": "Advanced",
  "ethnicity": "Indian",
  "lookingFor": "Relationship, Something Serious",
  "smoke": "No",
  "drink": "Socially",
  "kidCount": "Want someday",
  "netWorth": "₹25L - ₹50L"
}
```

- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Profile setup completed",
  "profile": {
    "userId": "usr_man_106",
    "name": "Arjun Sharma",
    "displayName": "Arjun",
    "gender": "man",
    "age": 24,
    "dob": "2002-05-14",
    "bio": "Software engineer in Bangalore. Coffee lover, weekend hiker.",
    "photos": [
      "https://storage.amara.com/users/usr_man_106/profile_primary.jpg"
    ],
    "language": "English, Hindi, Telugu",
    "appearance": "Athletic",
    "bodyType": "Fit",
    "height": 178,
    "englishLevel": "Advanced",
    "ethnicity": "Indian",
    "lookingFor": "Relationship, Something Serious",
    "smoke": "No",
    "drink": "Socially",
    "completion": 100
  }
}
```

2.5 Profile Completion Percentage

- **Endpoint**: `GET /profile/completion?userId=usr_man_106` OR `POST /profile/completion?userId=usr_man_106`

- **Headers**: `Authorization: Bearer <token>`
- **Success Response (200 OK)**:

```
100
```

(Or object):

```
{
  "completionPercentage": 100,
  "missingFields": []
}
```

---

3. Profile Management & Settings

3.1 Fetch Profile Details (Own Profile & Target User)

- **Endpoint**: `POST /profile/me?userId=usr_man_106` OR `GET /profile/me/{userId}`
- **Headers**: `Authorization: Bearer <token>`
- **Success Response (200 OK)**:

```
{
  "id": 106,
  "userId": "usr_man_106",
  "name": "Arjun Sharma",
  "displayName": "Arjun",
  "age": 24,
  "dob": "2002-05-14",
  "gender": "man",
  "bio": "Software engineer in Bangalore. Coffee lover, weekend hiker.",
  "photos": [
    "https://storage.amara.com/users/usr_man_106/photo_1.jpg",
    "https://storage.amara.com/users/usr_man_106/photo_2.jpg",
    "https://storage.amara.com/users/usr_man_106/photo_3.jpg"
  ],
  "profileImageUrl": "https://storage.amara.com/users/usr_man_106/photo_1.jpg",
  "telegramUsername": "arjun_dating",
  "verifiedSelfie": true,
  "selfieVerified": true,
  "language": "English, Hindi",
  "height": 178,
  "appearance": "Athletic",
  "bodyType": "Fit",
  "smoke": "No",
  "drink": "Socially",
  "englishLevel": "Advanced",
  "ethnicity": "Indian",
  "lookingFor": "Relationship",
  "currentCity": "Bangalore",
  "online": true,
  "completion": 100
}
```

Note on Privacy Guard: If viewing someone else's profile and the connection is NOT approved, `telegramUsername` must return `""` (empty string).

3.2 Update Full Profile

- **Endpoint**: `PUT /profile/update`

- **Headers**: `Authorization: Bearer <token>`
- **Request Body**:

```
{
  "userId": "usr_man_106",
  "displayName": "Arjun",
  "bio": "Passionate about photography & tech.",
  "dob": "2002-05-14",
  "height": 180,
  "language": "English, Hindi, German",
  "appearance": "Athletic",
  "bodyType": "Fit",
  "ethnicity": "Indian",
  "lookingFor": "Marriage, Long-term",
  "smoke": "Never",
  "drink": "Socially"
}
```

- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Profile updated successfully",
  "data": {
    "userId": "usr_man_106",
    "displayName": "Arjun",
    "bio": "Passionate about photography & tech."
  }
}
```

3.3 Update Basic Info (Name, Display Name, Bio, DOB)

- **Endpoint**: `PUT /profile/update-basic`
- **Request Body**:

```
{
  "userId": "usr_man_106",
  "name": "Arjun Sharma",
  "displayName": "Arjun",
  "bio": "Passionate about photography & tech.",
  "dob": "2002-05-14",
  "age": 24
}
```

- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Basic profile updated successfully"
}
```

3.4 Update Personal Details

- **Endpoint**: `PUT /profile/update-details`
- **Request Body**:

```
{
  "userId": "usr_man_106",
  "language": "English, Hindi",
  "bodyType": "Fit",
  "appearance": "Athletic",
  "height": 180,
  "englishLevel": "Advanced",
  "ethnicity": "Indian",
  "kidCount": "No kids",
  "netWorth": "₹25L - ₹50L"
}
```

- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Personal details updated successfully"
}
```

3.5 Update Preferences

- **Endpoint**: `PUT /profile/update-preferences`
- **Request Body**:

```
{
  "userId": "usr_man_106",
  "lookingFor": "Relationship, Something Serious",
  "smoke": "No",
  "drink": "Socially"
}
```

- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Preferences updated successfully"
}
```

3.6 Gallery Photos Management

- **Fetch Gallery**: `GET /users/{userId}/images`
- **Success Response (200 OK)**:

```
[
  {
    "id": 1,
    "userId": "usr_man_106",
    "imageUrl": "https://storage.amara.com/users/usr_man_106/img1.jpg",
    "isPrimary": true
  },
  {
    "id": 2,
    "userId": "usr_man_106",
    "imageUrl": "https://storage.amara.com/users/usr_man_106/img2.jpg",
    "isPrimary": false
  }
]
```

- **Set Primary Photo**: `PUT /users/{userId}/profile-photo/{imageId}`
- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Primary profile photo updated successfully"
}
```

- **Delete Photo**: `DELETE /users/{userId}/images/{imageId}`
- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Image deleted successfully"
}
```

---

4. Discovery, Home Feed & Search

4.1 Online Feed

Returns profiles currently active or online (default: opposite gender filter).

- **Endpoint**: `GET /dashboard/online?page=0&size=20&gender=woman`
- **Headers**: `Authorization: Bearer <token>`
- **Success Response (200 OK)**:

```
[
  {
    "id": 201,
    "userId": "usr_woman_201",
    "name": "Ananya Roy",
    "displayName": "Ananya",
    "age": 24,
    "gender": "woman",
    "bio": "Architect by day, espresso enthusiast by night.",
    "photos": [
      "https://images.unsplash.com/photo-1494790108377-be9c29b29330?w=800"
    ],
    "currentCity": "Mumbai",
    "online": true,
    "height": 168,
    "lookingFor": "Long-term, Marriage"
  }
]
```

4.2 Recent Profiles Feed

- **Endpoint**: `GET /dashboard/recent?page=0&size=20&gender=woman`
- **Success Response (200 OK)**:

```
[
  {
    "id": 202,
    "userId": "usr_woman_202",
    "name": "Pooja Hegde",
    "displayName": "Pooja",
    "age": 23,
    "gender": "woman",
    "bio": "Lover of travel and classical music.",
    "photos": [
      "https://images.unsplash.com/photo-1534528741775-53994a69daeb?w=800"
    ],
    "currentCity": "Delhi",
    "online": false,
    "height": 165
  }
]
```

4.3 Advanced Search

Filters candidate profiles by age, distance, appearance, body type, smoking, drinking, etc.

- **Endpoint**: `POST /search`
- **Headers**: `Authorization: Bearer <token>`
- **Request Body**:


```
{
  "minAge": 20,
  "maxAge": 30,
  "gender": ["Female"],
  "distance": 50,
  "appearance": ["Attractive", "Beautiful"],
  "bodyType": ["Slim", "Fit"],
  "languages": ["English", "Hindi"],
  "smoking": ["No", "Socially"],
  "drinking": ["Socially"],
  "page": 0,
  "size": 10
}
```


- **Success Response (200 OK)**:

```
{
  "content": [
    {
      "id": 201,
      "userId": "usr_woman_201",
      "name": "Ananya Roy",
      "displayName": "Ananya",
      "age": 24,
      "gender": "woman",
      "bio": "Architect by day, espresso enthusiast by night.",
      "photos": [
        "https://images.unsplash.com/photo-1494790108377-be9c29b29330?w=800"
      ],
      "currentCity": "Mumbai",
      "online": true,
      "height": 168
    }
  ],
  "totalElements": 1,
  "totalPages": 1,
  "number": 0,
  "size": 10
}
```


---

5. Dating Requests & Connections

5.1 Send Connection Request

A male user sends a connection / date request to a female user.

- **Endpoint**: `POST /connections/send`
- **Headers**: `Authorization: Bearer <token>`
- **Request Body**:


```
{
  "senderId": "usr_man_101",
  "receiverId": "usr_woman_201"
}
```


- **Success Response (200 OK / 201 Created)**:


```
{
  "status": "success",
  "message": "Connection request sent successfully",
  "request": {
    "id": "req_101_201",
    "senderId": "usr_man_101",
    "receiverId": "usr_woman_201",
    "status": "PENDING",
    "createdAt": "2026-10-13T16:15:00.000Z"
  }
}
```


- **Error Response (409 Conflict)**:


```
{
  "status": "error",
  "message": "A connection request is already pending between these users.",
  "code": "REQUEST_ALREADY_EXISTS"
}
```

5.2 Get Sent Requests

- **Endpoint**: `GET /connections/sent?userId=usr_man_101`
- **Success Response (200 OK)**:


```
[
  {
    "id": "req_101_201",
    "senderId": "usr_man_101",
    "receiverId": "usr_woman_201",
    "status": "PENDING",
    "createdAt": "2026-10-13T16:15:00.000Z",
    "receiver": {
      "userId": "usr_woman_201",
      "displayName": "Ananya",
      "age": 24,
      "gender": "woman",
      "photos": [
        "https://images.unsplash.com/photo-1494790108377-be9c29b29330?w=800"
      ]
    }
  }
]
```

5.3 Get Received Requests

- **Endpoint**: `GET /connections/received?userId=usr_woman_201`
- **Success Response (200 OK)**:


```
[
  {
    "id": "req_101_201",
    "senderId": "usr_man_101",
    "receiverId": "usr_woman_201",
    "status": "PENDING",
    "createdAt": "2026-10-13T16:15:00.000Z",
    "sender": {
      "userId": "usr_man_101",
      "displayName": "Rahul",
      "age": 25,
      "gender": "man",
      "photos": [
        "https://images.unsplash.com/photo-1500648767791-00dcc994a43e?w=800"
      ],
      "bio": "Coffee lover, marathon runner."
    }
  }
]
```

5.4 Accept Connection Request

When the female user accepts, the request status transitions to `APPROVED`, and the Telegram handle becomes unlocked.

- **Endpoint**: `PUT /connections/accept`
- **Headers**: `Authorization: Bearer <token>`
- **Request Body**:

```
{
  "requestId": "req_101_201",
  "receiverId": "usr_woman_201"
}
```

- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Request accepted. You are now connected!",
  "data": {
    "id": "req_101_201",
    "status": "APPROVED",
    "sender": {
      "userId": "usr_man_101",
      "displayName": "Rahul",
      "telegramUsername": "rahul_dating"
    },
    "receiver": {
      "userId": "usr_woman_201",
      "displayName": "Ananya",
      "telegramUsername": "ananya_dating"
    }
  }
}
```

5.5 Check Connection Status

- **Endpoint**: `GET /connections/status?senderId=usr_man_101&receiverId=usr_woman_201`
- **Success Response (200 OK)**:

```
{
  "status": "APPROVED",
  "requestId": "req_101_201"
}
```

(Values: `NONE`, `PENDING`, `APPROVED`, `REJECTED`)

---

6. Telegram Integration & Handoff

6.1 Connect Telegram Account

- **Endpoint**: `POST /telegram/connect`
- **Headers**: `Authorization: Bearer <token>`
- **Request (Body or Query Params)**:

```
{
  "userId": "usr_man_101",
  "username": "arjun_official"
}
```

- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Telegram connected successfully",
  "telegramUsername": "arjun_official"
}
```

6.2 Fetch Direct Telegram Link

- **Endpoint**: `GET /telegram/link?userId=usr_man_101&targetUserId=usr_woman_201`
- **Success Response (200 OK)**:

```
{
  "telegramUsername": "ananya_dating",
  "link": "https://t.me/ananya_dating"
}
```

- **Locked Error (403 Forbidden - if connection not approved)**:

```
{
  "status": "error",
  "message": "Telegram contact is locked until the connection request is approved.",
  "code": "CONNECTION_NOT_APPROVED"
}
```

6.3 Disconnect Telegram

- **Endpoint**: `DELETE /telegram/disconnect?userId=usr_man_101`
- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Telegram disconnected successfully",
  "telegramUsername": ""
}
```

---

7. Subscriptions & Payments (Razorpay)

7.1 Create Razorpay Order

- **Endpoint**: `POST /razorpay/create-order?userId=usr_man_101&plan=GOLD`
- **Headers**: `Authorization: Bearer <token>`
- **Success Response (200 OK)**:

```
{
  "status": "success",
  "orderId": "order_amara_987654321",
  "amount": 79900,
  "currency": "INR",
  "keyId": "rzp_test_1DP5mmOlF5G5ag",
  "plan": "GOLD"
}
```

7.2 Verify Razorpay Payment & Activate Subscription

- **Endpoint**: `POST /razorpay/verify`
- **Headers**: `Authorization: Bearer <token>`
- **Request Body**:

```
{
  "orderId": "order_amara_987654321",
  "paymentId": "pay_987654321",
  "signature": "e605d31481b4764835698b31a31d27976e...",
  "plan": "GOLD",
  "userId": "usr_man_101"
}
```

- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Payment verified and subscription activated successfully",
  "subscription": {
    "plan": "GOLD",
    "active": true,
    "startDate": "2026-10-13T16:00:00.000Z",
    "expiryDate": "2026-11-13T16:00:00.000Z",
    "remainingDays": 30
  }
}
```

7.3 Subscription Status & Remaining Days

- **Endpoint 1**: `GET /subscriber/status?userId=usr_man_101`
- **Success Response (200 OK)**:

```
{
  "active": true,
  "plan": "GOLD",
  "expiryDate": "2026-11-13T16:00:00.000Z"
}
```

- **Endpoint 2**: `GET /subscriber/remaining-days?userId=usr_man_101`
- **Success Response (200 OK)**:

```
30
```

---

8. Location & Proximity

8.1 Add User Location

- **Endpoint**: `POST /location/add`
- **Request Body**:

```
{
  "userId": "usr_man_101",
  "city": "Mumbai",
  "state": "Maharashtra",
  "country": "India",
  "latitude": 19.0760,
  "longitude": 72.8777
}
```

- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Location updated successfully"
}
```

8.2 Get Location History

- **Endpoint**: `GET /location/history/{userId}`
- **Success Response (200 OK)**:

```
[
  {
    "id": 1,
    "city": "Mumbai",
    "state": "Maharashtra",
    "country": "India",
    "createdAt": "2026-10-10T12:00:00.000Z"
  }
]
```

8.3 Get Nearby Profiles

- **Endpoint**: `GET /location/nearby?userId=usr_man_101&radius=50`
- **Success Response (200 OK)**:

```
[
  {
    "userId": "usr_woman_201",
    "displayName": "Ananya",
    "age": 24,
    "gender": "woman",
    "distanceKm": 4.2,
    "photos": [
      "https://images.unsplash.com/photo-1494790108377-be9c29b29330?w=800"
    ]
  }
]
```

---

9. Support Tickets & Help Center

9.1 Create Support Ticket

- **Endpoint**: `POST /support/create`
- **Request Body**:

```
{
  "userId": "usr_man_101",
  "subject": "Selfie Verification Query",
  "message": "My selfie verification is taking a few minutes, please check."
}
```

- **Success Response (201 Created)**:

```
{
  "status": "success",
  "message": "Support ticket created successfully",
  "ticket": {
    "id": 101,
    "userId": "usr_man_101",
    "subject": "Selfie Verification Query",
    "message": "My selfie verification is taking a few minutes, please check.",
    "status": "OPEN",
    "createdAt": "2026-10-13T16:20:00.000Z"
  }
}
```

9.2 Get My Tickets

- **Endpoint**: `GET /support/my?userId=usr_man_101`
- **Success Response (200 OK)**:

```
[
  {
    "id": 101,
    "subject": "Selfie Verification Query",
    "status": "OPEN",
    "createdAt": "2026-10-13T16:20:00.000Z"
  }
]
```

9.3 Close Ticket

- **Endpoint**: `PUT /support/close/{ticketId}?userId=usr_man_101`
- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Support ticket closed successfully",
  "ticket": {
    "id": 101,
    "status": "CLOSED"
  }
}
```

---

10. Privacy, Safety & User Reporting

10.1 Accept Privacy Policy

- **Endpoint**: `POST /privacy/accept?userId=usr_man_101`
- **Success Response (200 OK)**:

```
{
  "status": "success",
  "accepted": true
}
```

10.2 Check Privacy Policy Acceptance Status

- **Endpoint**: `GET /privacy/status?userId=usr_man_101`
- **Success Response (200 OK)**:

```
{
  "accepted": true
}
```

(Or boolean `true`)

10.3 Report a Profile

- **Endpoint**: `POST /reports/report`
- **Request Body**:

```
{
  "reporterUserId": "usr_man_101",
  "reportedUserId": "usr_woman_207",
  "reason": "SPAM_SCAM",
  "details": "User sent suspicious commercial telegram links."
}
```

(Valid reasons: `FAKE_PROFILE`, `HARASSMENT`, `INAPPROPRIATE_CONTENT`, `SPAM_SCAM`, `SAFETY_CONCERN`)

- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Report submitted successfully. Our safety team will review it within 24 hours.",
  "data": {
    "id": 501,
    "reporterUserId": "usr_man_101",
    "reportedUserId": "usr_woman_207",
    "reason": "SPAM_SCAM",
    "status": "PENDING"
  }
}
```

10.4 Presence & Online Status

- **Set Online**: `PUT /status/online?userId=usr_man_101`
- **Success Response (200 OK)**: `{"status": "online"}`
- **Set Offline**: `PUT /status/offline?userId=usr_man_101`
- **Success Response (200 OK)**: `{"status": "offline"}`

10.5 In-App Notifications

- **Endpoint**: `GET /notification?userId=usr_man_101`

- **Success Response (200 OK)**:

```
[
  {
    "id": 1,
    "title": "New Date Request",
    "message": "Ananya approved your connection request!",
    "type": "CONNECTION_ACCEPTED",
    "read": false,
    "createdAt": "2026-10-13T16:15:00.000Z"
  }
]
```

10.6 Notification Preferences

Allows users to retrieve and update their notification delivery settings (Push, Email, Match Alerts). When `emailNotifications` is set to `false`, social notifications (such as new connection requests and weekly match digests) will NOT be dispatched to the user's email address. Critical security and financial transactional emails (Registration OTP, Password Reset OTP, and Subscription Invoices) are always delivered for account security and compliance.

10.6.1 Get Notification Preferences

- **Endpoint**: `GET /notification/preferences`
- **Auth Required**: Yes (Bearer Token)
- **Query Parameters**:
- `userId` (optional string): Target user ID (defaults to authenticated token user).
- **Success Response (200 OK)**:

```
{
  "status": "success",
  "data": {
    "emailNotifications": true,
    "pushNotifications": true,
    "matchAlerts": true
  }
}
```

10.6.2 Update Notification Preferences

- **Endpoint**: `PUT /notification/preferences`
- **Auth Required**: Yes (Bearer Token)
- **Request Body**:

```
{
  "emailNotifications": false,
  "pushNotifications": true,
  "matchAlerts": true
}
```

- **Success Response (200 OK)**:

```
{
  "status": "success",
  "message": "Notification preferences updated successfully",
  "data": {
    "emailNotifications": false,
    "pushNotifications": true,
    "matchAlerts": true
  }
}
```

---

11. Transactional & Notification Email Templates (HTML & Plain-Text)

This section contains all production-grade, responsive HTML email templates and their plain-text counterparts used across the AMARA platform. Backend developers can directly copy these templates into their mailing engine (e.g., SendGrid, AWS SES, Resend, or Nodemailer) and replace the dynamic mustache tokens (`{{variable_name}}`) at dispatch time.

Email Dispatch Policy & Preference Matrix

Template #	Template Name	Triggering Endpoint	Category	Preference Check
Template 1	Registration Email OTP	`POST /register`	Transactional / Security	**Always Send** (Critical Auth)
Template 2	Password Reset OTP	`POST /auth/forgot-password/send-otp`	Transactional / Security	**Always Send** (Critical Auth)
Template 3	Welcome & Account Verified	`POST /verify-register/otp` / Onboarding Complete	Onboarding	**Always Send** (Account Activation)
Template 4	New Connection Request	`POST /connections/send`	Social Notification	**Check** `emailNotifications === true`
Template 5	Connection Accepted (It's a Match!)	`PUT /connections/accept`	Social Notification	**Check** `emailNotifications === true`
Template 6	Subscription & Invoice Confirmation	`POST /razorpay/verify`	Financial / Transactional	**Always Send** (Payment Receipt)

11.1 Template 1: Registration Email OTP Verification

- **Trigger**: `POST /register`
- **Category**: Transactional / Security (Bypasses preference toggle)
- **Subject**: `Amara - Verify your email address (Code: {{otp}})`
- **Dynamic Variables**:
- `{{name}}`: User's full name or display name.
- `{{otp}}`: 6-digit numeric verification code.
- `{{expiryMinutes}}`: Expiration duration in minutes (default: `10`).

Responsive HTML Template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
```

```
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Verify Your Email - Amara</title>
  <style>
    body { margin: 0; padding: 0; background-color: #0F051D; font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", F
    table { border-spacing: 0; }
    td { padding: 0; }
    img { border: 0; }
    .wrapper { width: 100%; table-layout: fixed; background-color: #0F051D; padding-bottom: 40px; }
    .main-table { background-color: #19092F; margin: 0 auto; width: 100%; max-width: 600px; border-radius: 16px; border: 1
    .header { background: linear-gradient(135deg, #7C3AED 0%, #4C1D95 100%); padding: 36px 20px; text-align: center; }
    .brand-title { font-size: 30px; font-weight: 800; letter-spacing: 4px; color: #FFFFFF; margin: 0; text-transform: uppe
    .brand-subtitle { font-size: 13px; color: #E9D5FF; letter-spacing: 1px; margin-top: 6px; }
    .content { padding: 36px 32px; }
    .greeting { font-size: 20px; font-weight: 700; color: #FFFFFF; margin-bottom: 12px; }
    .body-text { font-size: 15px; line-height: 24px; color: #D1D5DB; margin-bottom: 24px; }
    .otp-card { background-color: #240E42; border: 2px dashed #8B5CF6; border-radius: 12px; padding: 24px; text-align: cen
    .otp-label { font-size: 12px; letter-spacing: 1.5px; text-transform: uppercase; color: #C4B5FD; font-weight: 600; marg
    .otp-code { font-size: 38px; font-weight: 800; letter-spacing: 8px; color: #FFFFFF; font-family: "Courier New", Courie
    .expiry-note { font-size: 13px; color: #9CA3AF; margin-top: 10px; }
    .security-notice { background-color: #130623; border-left: 4px solid #7C3AED; padding: 14px 16px; border-radius: 6px;
    .footer { text-align: center; padding: 24px 32px; font-size: 12px; color: #6B7280; line-height: 18px; }
    .footer a { color: #A78BFA; text-decoration: none; }
  </style>
</head>
<body>
  <div class="wrapper">
    <table class="main-table" align="center">
      <tr>
        <td class="header">
          <h1 class="brand-title">AMARA</h1>
          <div class="brand-subtitle">EXCLUSIVELY CURATED DATING</div>
        </td>
      </tr>
      <tr>
        <td class="content">
          <div class="greeting">Hello {{name}},</div>
          <div class="body-text">
            Welcome to AMARA! To complete your registration and activate your secure account, please use the 6-digit verif
          </div>
          <div class="otp-card">
            <div class="otp-label">Your Email Verification Code</div>
            <div class="otp-code">{{otp}}</div>
            <div class="expiry-note">    This code expires in {{expiryMinutes}} minutes</div>
          </div>
          <div class="body-text">
            Enter this code in the AMARA mobile app to verify your email address. Do not share this code with anyone.
          </div>
          <div class="security-notice">
              <strong>Security Tip:</strong> AMARA employees will never ask you for your verification code or password. I
          </div>
        </td>
      </tr>
      <tr>
        <td class="footer">
          &copy; 2026 AMARA Dating Inc. All rights reserved.<br>
          This is an automated security transmission sent to your registered email address.<br>
          <a href="https://amara.com/privacy">Privacy Policy</a> &bull; <a href="https://amara.com/terms">Terms of Service
        </td>
      </tr>
    </table>
  </div>
</body>
</html>
```

Plain-Text Fallback:

```
AMARA - Exclusively Curated Dating

Hello {{name}},

Welcome to AMARA! To complete your registration and verify your account, please enter the following 6-digit code in the ap

VERIFICATION CODE: {{otp}}
(Valid for {{expiryMinutes}} minutes)

Do not share this code with anyone. AMARA representatives will never ask for your verification code.

If you did not initiate this request, you can safely ignore this email.

Best regards,
The AMARA Security Team
https://amara.com
```

11.2 Template 2: Password Reset OTP

- **Trigger**: `POST /auth/forgot-password/send-otp`
- **Category**: Transactional / Security (Bypasses preference toggle)
- **Subject**: `Amara - Password Reset Code (Code: {{otp}})`
- **Dynamic Variables**:
- `{{name}}`: User's name (or `Member`).
- `{{otp}}`: 6-digit password reset code.
- `{{expiryMinutes}}`: Expiration duration in minutes (default: `10`).

Responsive HTML Template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Reset Your Password - Amara</title>
  <style>
    body { margin: 0; padding: 0; background-color: #0F051D; font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", F
    table { border-spacing: 0; }
    td { padding: 0; }
    .wrapper { width: 100%; table-layout: fixed; background-color: #0F051D; padding-bottom: 40px; }
    .main-table { background-color: #19092F; margin: 0 auto; width: 100%; max-width: 600px; border-radius: 16px; border: 1
    .header { background: linear-gradient(135deg, #DC2626 0%, #7C3AED 100%); padding: 36px 20px; text-align: center; }
    .brand-title { font-size: 30px; font-weight: 800; letter-spacing: 4px; color: #FFFFFF; margin: 0; text-transform: uppe
    .brand-subtitle { font-size: 13px; color: #FECACA; letter-spacing: 1px; margin-top: 6px; }
    .content { padding: 36px 32px; }
    .greeting { font-size: 20px; font-weight: 700; color: #FFFFFF; margin-bottom: 12px; }
    .body-text { font-size: 15px; line-height: 24px; color: #D1D5DB; margin-bottom: 24px; }
    .otp-card { background-color: #260E2A; border: 2px dashed #EF4444; border-radius: 12px; padding: 24px; text-align: cen
    .otp-label { font-size: 12px; letter-spacing: 1.5px; text-transform: uppercase; color: #FCA5A5; font-weight: 600; marg
    .otp-code { font-size: 38px; font-weight: 800; letter-spacing: 8px; color: #FFFFFF; font-family: "Courier New", Courie
    .expiry-note { font-size: 13px; color: #9CA3AF; margin-top: 10px; }
    .security-notice { background-color: #130623; border-left: 4px solid #EF4444; padding: 14px 16px; border-radius: 6px;
    .footer { text-align: center; padding: 24px 32px; font-size: 12px; color: #6B7280; line-height: 18px; }
    .footer a { color: #A78BFA; text-decoration: none; }
  </style>
</head>
<body>
  <div class="wrapper">
    <table class="main-table" align="center">
      <tr>
        <td class="header">
```

```
          <h1 class="brand-title">AMARA</h1>
          <div class="brand-subtitle">ACCOUNT SECURITY & PASSWORD RECOVERY</div>
        </td>
      </tr>
      <tr>
        <td class="content">
          <div class="greeting">Hello {{name}},</div>
          <div class="body-text">
            We received a request to reset the password for your AMARA account. Use the one-time code below to proceed wit
          </div>
          <div class="otp-card">
            <div class="otp-label">Password Reset One-Time Code</div>
            <div class="otp-code">{{otp}}</div>
            <div class="expiry-note">    Expires in {{expiryMinutes}} minutes</div>
          </div>
          <div class="body-text">
            After entering this code in the AMARA app, you will be able to choose a new strong password.
          </div>
          <div class="security-notice">
               <strong>Didn't request this?</strong> If you didn't ask to reset your password, someone may have entered y
          </div>
        </td>
      </tr>
      <tr>
        <td class="footer">
          &copy; 2026 AMARA Dating Inc. All rights reserved.<br>
          Automated security dispatch sent to your verified email.<br>
          <a href="https://amara.com/privacy">Privacy Policy</a> &bull; <a href="https://amara.com/support">Contact Suppor
        </td>
      </tr>
    </table>
  </div>
</body>
</html>
```

Plain-Text Fallback:

```
AMARA - Password Reset Request

Hello {{name}},

We received a request to reset your AMARA account password.

Your one-time reset code is: {{otp}}
(This code will expire in {{expiryMinutes}} minutes)

Enter this code in the AMARA mobile app to create a new password.

If you did not request a password reset, please ignore this email. Your account remains safe.

Best regards,
The AMARA Security Team
https://amara.com
```

11.3 Template 3: Welcome & Account Verified

- **Trigger**: `POST /verify-register/otp` (Onboarding completed)
- **Category**: Onboarding & Activation
- **Subject**: `Welcome to Amara, {{name}}!`
- **Dynamic Variables**:
- `{{name}}`: User's display name.

- `{{appDeepLink}}`: Deep link URL to open the home feed (e.g., `amara://home`).

Responsive HTML Template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Welcome to Amara</title>
  <style>
    body { margin: 0; padding: 0; background-color: #0F051D; font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", F
    table { border-spacing: 0; }
    td { padding: 0; }
    .wrapper { width: 100%; table-layout: fixed; background-color: #0F051D; padding-bottom: 40px; }
    .main-table { background-color: #19092F; margin: 0 auto; width: 100%; max-width: 600px; border-radius: 16px; border: 1
    .header { background: linear-gradient(135deg, #7C3AED 0%, #C026D3 100%); padding: 40px 20px; text-align: center; }
    .brand-title { font-size: 32px; font-weight: 800; letter-spacing: 4px; color: #FFFFFF; margin: 0; }
    .brand-subtitle { font-size: 14px; color: #F5D0FE; letter-spacing: 1.5px; margin-top: 8px; font-weight: 500; }
    .content { padding: 36px 32px; }
    .greeting { font-size: 22px; font-weight: 700; color: #FFFFFF; margin-bottom: 14px; }
    .body-text { font-size: 15px; line-height: 24px; color: #D1D5DB; margin-bottom: 20px; }
    .feature-card { background-color: #240E42; border-radius: 12px; padding: 20px; margin-bottom: 16px; border: 1px solid
    .feature-title { font-size: 16px; font-weight: 700; color: #E9D5FF; margin-bottom: 6px; }
    .feature-desc { font-size: 14px; color: #9CA3AF; line-height: 20px; margin: 0; }
    .cta-container { text-align: center; margin: 32px 0; }
    .cta-button { display: inline-block; background: linear-gradient(135deg, #7C3AED 0%, #6D28D9 100%); color: #FFFFFF; te
    .footer { text-align: center; padding: 24px 32px; font-size: 12px; color: #6B7280; line-height: 18px; }
    .footer a { color: #A78BFA; text-decoration: none; }
  </style>
</head>
<body>
  <div class="wrapper">
    <table class="main-table" align="center">
      <tr>
        <td class="header">
          <h1 class="brand-title">AMARA</h1>
          <div class="brand-subtitle">WHERE ELEGANCE MEETS CONNECTION</div>
        </td>
      </tr>
      <tr>
        <td class="content">
          <div class="greeting">Welcome to the community, {{name}}!  </div>
          <div class="body-text">
            Your AMARA profile is verified and active. You are now part of an exclusive space designed for genuine, high-i
          </div>

          <div class="feature-card">
            <div class="feature-title">✨ Complete Your Visual Story</div>
            <p class="feature-desc">Add at least 3 high-quality photos and your verified selfie badge to achieve up to 4x
          </div>

          <div class="feature-card">
            <div class="feature-title">✈  Seamless Telegram Handoff</div>
            <p class="feature-desc">Connect your Telegram username so approved connections can start conversations instant
          </div>

          <div class="feature-card">
            <div class="feature-title">   Safety & Privacy First</div>
            <p class="feature-desc">Your privacy is paramount. Phone numbers and email addresses are always kept private f
          </div>

          <div class="cta-container">
            <a href="{{appDeepLink}}" class="cta-button">Start Exploring Matches</a>
          </div>
        </td>
      </tr>
```

```
      <tr>
        <td class="footer">
          &copy; 2026 AMARA Dating Inc. All rights reserved.<br>
          Sent with care to {{name}}.<br>
          <a href="https://amara.com/guidelines">Community Guidelines</a> &bull; <a href="https://amara.com/privacy">Priva
        </td>
      </tr>
    </table>
  </div>
</body>
</html>
```

Plain-Text Fallback:

```
Welcome to AMARA, {{name}}!

Your profile is officially verified and active.

Top Tips to Make the Most of Amara:
1. Complete your visual story: Upload at least 3 photos to stand out.
2. Verified Selfie: Complete the quick biometric verification to earn your trusted verified checkmark.
3. Link Telegram: Enable instant secure communication with approved matches.

Open the AMARA app now to discover your first connections:
https://amara.com/app

Best regards,
The AMARA Team
```

11.4 Template 4: New Connection Request Received

- **Trigger**: `POST /connections/send`
- **Category**: Social Notification
- **Preference Check**: **Dispatch ONLY IF recipient has** `emailNotifications === true` (configured via `PUT /notification/preferences`).
- **Subject**: `New Connection Request from {{senderName}} on Amara`
- **Dynamic Variables**:
- `{{recipientName}}`: Receiver's display name.
- `{{senderName}}`: Sender's display name.
- `{{senderAge}}`: Sender's calculated age.
- `{{senderCity}}`: Sender's location city.
- `{{senderPhotoUrl}}`: Primary photo URL of sender.
- `{{reviewDeepLink}}`: Deep link URL directly to received requests tab (`amara://requests/received`).

Responsive HTML Template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>New Connection Request - Amara</title>
  <style>
    body { margin: 0; padding: 0; background-color: #0F051D; font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", F
    table { border-spacing: 0; }
    td { padding: 0; }
    .wrapper { width: 100%; table-layout: fixed; background-color: #0F051D; padding-bottom: 40px; }
    .main-table { background-color: #19092F; margin: 0 auto; width: 100%; max-width: 600px; border-radius: 16px; border: 1
    .header { background: linear-gradient(135deg, #7C3AED 0%, #4C1D95 100%); padding: 32px 20px; text-align: center; }
```

```
    .brand-title { font-size: 28px; font-weight: 800; letter-spacing: 4px; color: #FFFFFF; margin: 0; }
    .content { padding: 36px 32px; }
    .greeting { font-size: 20px; font-weight: 700; color: #FFFFFF; margin-bottom: 12px; }
    .body-text { font-size: 15px; line-height: 24px; color: #D1D5DB; margin-bottom: 24px; }
    .profile-card { background: #230E40; border: 1px solid #431E7D; border-radius: 16px; padding: 24px; text-align: center
    .avatar-img { width: 110px; height: 110px; border-radius: 55px; object-fit: cover; border: 3px solid #8B5CF6; margin-b
    .profile-name { font-size: 20px; font-weight: 800; color: #FFFFFF; margin: 0; }
    .profile-meta { font-size: 14px; color: #C4B5FD; margin-top: 4px; font-weight: 500; }
    .cta-container { text-align: center; margin: 28px 0 16px; }
    .cta-button { display: inline-block; background: linear-gradient(135deg, #7C3AED 0%, #6D28D9 100%); color: #FFFFFF; te
    .footer { text-align: center; padding: 24px 32px; font-size: 12px; color: #6B7280; line-height: 18px; }
    .footer a { color: #A78BFA; text-decoration: none; }
  </style>
</head>
<body>
  <div class="wrapper">
    <table class="main-table" align="center">
      <tr>
        <td class="header">
          <h1 class="brand-title">AMARA</h1>
        </td>
      </tr>
      <tr>
        <td class="content">
          <div class="greeting">Hi {{recipientName}},</div>
          <div class="body-text">
            Someone special wants to connect with you! You have received a new connection invitation on AMARA:
          </div>

          <div class="profile-card">
            <img src="{{senderPhotoUrl}}" alt="{{senderName}}" class="avatar-img">
            <h2 class="profile-name">{{senderName}}, {{senderAge}}</h2>
            <div class="profile-meta">   {{senderCity}}</div>
            <div class="cta-container">
              <a href="{{reviewDeepLink}}" class="cta-button">Review Request & Reply</a>
            </div>
          </div>

          <div class="body-text" style="font-size: 13px; color: #9CA3AF; text-align: center;">
            Accepting the request will allow you both to view Telegram contact info and begin chatting.
          </div>
        </td>
      </tr>
      <tr>
        <td class="footer">
          You received this email because email notifications are turned ON in your AMARA settings.<br>
          To change this, open the AMARA app &rarr; Profile &rarr; Preferences &rarr; <strong>Toggle Email Notifications</
          <a href="https://amara.com/privacy">Privacy Policy</a> &bull; <a href="https://amara.com/preferences">Manage Not
        </td>
      </tr>
    </table>
  </div>
</body>
</html>
```

Plain-Text Fallback:

```
Hi {{recipientName}},

You have received a new connection request on AMARA from {{senderName}}, {{senderAge}} ({{senderCity}})!

Open the AMARA app to review their profile and approve or decline the invitation:
{{reviewDeepLink}}

---
You are receiving this email because Email Notifications are enabled in your AMARA Preferences. You can disable this anyti
```

11.5 Template 5: Connection Request Accepted (It's a Match!)

- **Trigger**: `PUT /connections/accept`
- **Category**: Social Notification
- **Preference Check**: **Dispatch ONLY IF sender has** `emailNotifications === true` (configured via `PUT /notification/preferences`).
- **Subject**: `It's a Match! {{partnerName}} accepted your request on Amara`
- **Dynamic Variables**:
- `{{userName}}`: User's display name.
- `{{partnerName}}`: Match partner's display name.
- `{{partnerTelegram}}`: Match partner's Telegram username (without @).
- `{{partnerPhotoUrl}}`: Partner's primary photo.
- `{{telegramChatUrl}}`: Direct Telegram link (`https://t.me/{{partnerTelegram}}`).

Responsive HTML Template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>It's a Match! - Amara</title>
  <style>
    body { margin: 0; padding: 0; background-color: #0F051D; font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", F
    table { border-spacing: 0; }
    td { padding: 0; }
    .wrapper { width: 100%; table-layout: fixed; background-color: #0F051D; padding-bottom: 40px; }
    .main-table { background-color: #19092F; margin: 0 auto; width: 100%; max-width: 600px; border-radius: 16px; border: 1
    .header { background: linear-gradient(135deg, #10B981 0%, #7C3AED 100%); padding: 36px 20px; text-align: center; }
    .brand-title { font-size: 28px; font-weight: 800; letter-spacing: 4px; color: #FFFFFF; margin: 0; }
    .badge { display: inline-block; background-color: rgba(255, 255, 255, 0.2); padding: 4px 14px; border-radius: 20px; fo
    .content { padding: 36px 32px; }
    .greeting { font-size: 22px; font-weight: 700; color: #FFFFFF; margin-bottom: 12px; }
    .body-text { font-size: 15px; line-height: 24px; color: #D1D5DB; margin-bottom: 24px; }
    .match-card { background: linear-gradient(145deg, #240E42, #18072E); border: 2px solid #8B5CF6; border-radius: 16px; p
    .partner-avatar { width: 120px; height: 120px; border-radius: 60px; object-fit: cover; border: 4px solid #10B981; marg
    .match-name { font-size: 22px; font-weight: 800; color: #FFFFFF; margin: 0; }
    .telegram-box { background-color: #0088CC20; border: 1px solid #0088CC; border-radius: 8px; padding: 12px; margin: 20p
    .cta-container { text-align: center; margin-top: 20px; }
    .cta-telegram { display: inline-block; background: #0088CC; color: #FFFFFF; text-decoration: none; font-weight: 700; f
    .footer { text-align: center; padding: 24px 32px; font-size: 12px; color: #6B7280; line-height: 18px; }
    .footer a { color: #A78BFA; text-decoration: none; }
  </style>
</head>
<body>
  <div class="wrapper">
    <table class="main-table" align="center">
      <tr>
        <td class="header">
          <h1 class="brand-title">AMARA</h1>
```

```
          <div class="badge">   MUTUAL MATCH</div>
        </td>
      </tr>
      <tr>
        <td class="content">
          <div class="greeting">Wonderful news, {{userName}}!</div>
          <div class="body-text">
            <strong>{{partnerName}}</strong> has reviewed your dating request and accepted it! You are now mutually connec
          </div>

          <div class="match-card">
            <img src="{{partnerPhotoUrl}}" alt="{{partnerName}}" class="partner-avatar">
            <h2 class="match-name">{{partnerName}}</h2>
            <div class="telegram-box">
              ✈  Telegram: @{{partnerTelegram}}
            </div>
            <div class="cta-container">
              <a href="{{telegramChatUrl}}" class="cta-telegram">Open Chat on Telegram</a>
            </div>
          </div>

          <div class="body-text" style="font-size: 14px; color: #9CA3AF; text-align: center;">
            Say hello and strike up a conversation. Remember to keep communication respectful and kind.
          </div>
        </td>
      </tr>
      <tr>
        <td class="footer">
          You received this match notification because email notifications are active in your preferences.<br>
          <a href="https://amara.com/privacy">Privacy Policy</a> &bull; <a href="https://amara.com/safety">Dating Safety T
        </td>
      </tr>
    </table>
  </div>
</body>
</html>
```

Plain-Text Fallback:

```
It's a Match!

Hello {{userName}},

Great news! {{partnerName}} accepted your connection request on AMARA.

You can now connect directly via Telegram:
Telegram Username: @{{partnerTelegram}}
Direct Chat Link: {{telegramChatUrl}}

Have a wonderful conversation!

Best regards,
The AMARA Team
```

11.6 Template 6: Subscription & Payment Confirmation

- **Trigger**: `POST /razorpay/verify`
- **Category**: Financial / Transactional (Always sent for billing compliance)
- **Subject**: `Your Amara {{planName}} Membership is Active! (Order: {{orderId}})`
- **Dynamic Variables**:
- `{{userName}}`: Subscriber's name.

- `{{planName}}`: Name of tier (e.g. `Gold`, `VIP`, `Platinum`).
- `{{amountPaid}}`: Formatted currency amount (e.g. `₹1,999`).
- `{{orderId}}`: Razorpay order identifier.
- `{{paymentId}}`: Razorpay transaction payment ID.
- `{{startDate}}`: Subscription start date.
- `{{endDate}}`: Subscription expiration date.
- `{{dailyRequestLimit}}`: Daily request limit (e.g. `20` or `Unlimited`).

Responsive HTML Template:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Subscription Confirmation - Amara</title>
  <style>
    body { margin: 0; padding: 0; background-color: #0F051D; font-family: -apple-system, BlinkMacSystemFont, "Segoe UI", F
    table { border-spacing: 0; width: 100%; }
    td { padding: 0; }
    .wrapper { width: 100%; table-layout: fixed; background-color: #0F051D; padding-bottom: 40px; }
    .main-table { background-color: #19092F; margin: 0 auto; width: 100%; max-width: 600px; border-radius: 16px; border: 1
    .header { background: linear-gradient(135deg, #F59E0B 0%, #7C3AED 100%); padding: 36px 20px; text-align: center; }
    .brand-title { font-size: 30px; font-weight: 800; letter-spacing: 4px; color: #FFFFFF; margin: 0; }
    .plan-badge { display: inline-block; background: rgba(0, 0, 0, 0.3); padding: 6px 18px; border-radius: 20px; font-size
    .content { padding: 36px 32px; }
    .greeting { font-size: 20px; font-weight: 700; color: #FFFFFF; margin-bottom: 12px; }
    .body-text { font-size: 15px; line-height: 24px; color: #D1D5DB; margin-bottom: 24px; }
    .receipt-table { width: 100%; background-color: #230E40; border-radius: 12px; border: 1px solid #3D186D; margin: 20px
    .receipt-row td { padding: 12px 18px; font-size: 14px; border-bottom: 1px solid #2F1354; }
    .receipt-row:last-child td { border-bottom: none; }
    .receipt-label { color: #9CA3AF; font-weight: 500; }
    .receipt-value { color: #FFFFFF; font-weight: 700; text-align: right; }
    .total-row td { padding: 16px 18px; font-size: 16px; background-color: #2C1250; }
    .total-label { color: #FDE68A; font-weight: 700; }
    .total-value { color: #FDE68A; font-weight: 800; text-align: right; font-size: 18px; }
    .footer { text-align: center; padding: 24px 32px; font-size: 12px; color: #6B7280; line-height: 18px; }
    .footer a { color: #A78BFA; text-decoration: none; }
  </style>
</head>
<body>
  <div class="wrapper">
    <table class="main-table" align="center">
      <tr>
        <td class="header">
          <h1 class="brand-title">AMARA</h1>
          <div class="plan-badge">   {{planName}} MEMBERSHIP ACTIVE</div>
        </td>
      </tr>
      <tr>
        <td class="content">
          <div class="greeting">Thank you, {{userName}}!</div>
          <div class="body-text">
            Your payment was successful and your premium membership has been unlocked. Here is your official receipt:
          </div>

          <table class="receipt-table">
            <tr class="receipt-row">
              <td class="receipt-label">Membership Tier</td>
              <td class="receipt-value">{{planName}} Plan</td>
            </tr>
            <tr class="receipt-row">
              <td class="receipt-label">Order ID</td>
              <td class="receipt-value">{{orderId}}</td>
            </tr>
```

```
            <tr class="receipt-row">
              <td class="receipt-label">Payment ID</td>
              <td class="receipt-value">{{paymentId}}</td>
            </tr>
            <tr class="receipt-row">
              <td class="receipt-label">Start Date</td>
              <td class="receipt-value">{{startDate}}</td>
            </tr>
            <tr class="receipt-row">
              <td class="receipt-label">Valid Until</td>
              <td class="receipt-value">{{endDate}}</td>
            </tr>
            <tr class="receipt-row">
              <td class="receipt-label">Daily Dating Request Limit</td>
              <td class="receipt-value">{{dailyRequestLimit}} / day</td>
            </tr>
            <tr class="total-row">
              <td class="total-label">Total Amount Paid</td>
              <td class="total-value">{{amountPaid}}</td>
            </tr>
          </table>

          <div class="body-text" style="font-size: 14px; color: #9CA3AF; margin-top: 20px;">
            Enjoy unlimited profile browsing, verified contact unlocks, and enhanced discovery visibility.
          </div>
        </td>
      </tr>
      <tr>
        <td class="footer">
          &copy; 2026 AMARA Dating Inc. &bull; Tax Invoice & Payment Confirmation<br>
          For billing questions, please contact our support team at <a href="mailto:billing@amara.com">billing@amara.com</
        </td>
      </tr>
    </table>
  </div>
</body>
</html>
```

Plain-Text Fallback:

```
AMARA - Membership & Payment Confirmation

Thank you, {{userName}}!

Your {{planName}} subscription is now ACTIVE.

Invoice Summary:
- Plan: {{planName}}
- Order ID: {{orderId}}
- Payment ID: {{paymentId}}
- Active From: {{startDate}}
- Valid Until: {{endDate}}
- Daily Request Limit: {{dailyRequestLimit}} per day
- Amount Paid: {{amountPaid}}

Thank you for choosing AMARA!

Best regards,
The AMARA Billing Team
https://amara.com
```

11.7 Backend Mailer Service Implementation Guide (Node.js / TypeScript)

Here is the recommended production implementation for the backend mailing service using **Nodemailer** (configured

for **Gmail SMTP**, SendGrid, or AWS SES). This snippet strictly enforces the **Glambu Preference Check Rule** before dispatching notifications.

```
import nodemailer from "nodemailer";
import { User, NotificationPreferences } from "./models";

// Transport configuration (defaults to Gmail SMTP for zero-overhead deployment)
const transporter = nodemailer.createTransport(
  process.env.SMTP_SERVICE === "gmail" || !process.env.SMTP_HOST
    ? {
        service: "gmail",
        auth: {
          user: process.env.SMTP_USER, // e.g. "amara.dating.app@gmail.com"
          pass: process.env.SMTP_PASS, // 16-character Google App Password (not normal password)
        },
      }
    : {
        host: process.env.SMTP_HOST,
        port: Number(process.env.SMTP_PORT) || 587,
        secure: process.env.SMTP_SECURE === "true",
        auth: {
          user: process.env.SMTP_USER,
          pass: process.env.SMTP_PASS,
        },
      }
);

interface SendEmailOptions {
  to: string;
  userId?: string;
  subject: string;
  html: string;
  text: string;
  category: "TRANSACTIONAL" | "NOTIFICATION";
}

export async function sendAmaraEmail(options: SendEmailOptions): Promise<boolean> {
  const { to, userId, subject, html, text, category } = options;

  // 1. GLAMBU PREFERENCE CHECK:
  // If this is a social notification / invite reminder / digest, verify user hasn't turned off email notifications
  if (category === "NOTIFICATION" && userId) {
    const prefs = await NotificationPreferences.findOne({ userId });
    if (prefs && prefs.emailNotifications === false) {
      console.log(`[Mailer]     Skipped notification email to ${to} — user disabled email notifications in preferences.`);
      return false;
    }
  }

  // 2. Dispatch via SMTP
  try {
    const info = await transporter.sendMail({
      from: process.env.SMTP_FROM || `"AMARA Dating" <${process.env.SMTP_USER}>`,
      to,
      subject,
      text,
      html,
    });
    console.log(`[Mailer] ✉  Successfully sent "${subject}" to ${to} (MessageId: ${info.messageId})`);
    return true;
  } catch (err) {
    console.error(`[Mailer] ✖ Failed to dispatch email to ${to}:`, err);
    throw err;
  }
}
```

11.8 Gmail SMTP Setup Guide (Zero-Configuration Deployment)

For initial production and staging deployments, **Gmail SMTP** is the easiest and most cost-effective option because it requires no domain DNS validation, credit card, or third-party email service approval.

1. Generate a 16-Character Google App Password (60 seconds)

> **IMPORTANT**: Google accounts with 2-Step Verification do **not** accept your regular account password for SMTP. You must generate a dedicated 16-character **App Password**:

1. Log into your Google Account: **myaccount.google.com/security**.
2. Under "*How you sign in to Google*", ensure **2-Step Verification** is turned **ON**.
3. Click into **2-Step Verification**, scroll to the bottom, and select **App passwords**.
4. In the "*App name*" box, enter `AMARA Dating Backend`.
5. Click **Create**. Google will display a 16-character code (e.g. `abcd efgh ijkl mnop`).
6. Copy this password (without spaces).

2. Backend Environment Variables (`.env`)

Add these variables to your production backend `.env` file:

```
# ------------------------------------------------
# GMAIL SMTP CONFIGURATION (Amara Dating Backend)
# ------------------------------------------------
SMTP_SERVICE=gmail
SMTP_HOST=smtp.gmail.com
SMTP_PORT=465
SMTP_SECURE=true
SMTP_USER=amara.dating.app@gmail.com
SMTP_PASS=abcdefghijklmnop               # 16-character Google App Password
SMTP_FROM="Amara Dating <amara.dating.app@gmail.com>"
```

3. Automatic Trigger Matrix (Glambu Behavior)

Event / Action	Endpoint	Template Used	Category	Rule Enforced
New Dating Request	`POST /connections/send`	Template 4 (Connection Request)	`NOTIFICATION`	**Dispatch ONLY IF recipient's** `emailNotifications === true`
Request Approved	`PUT /connections/accept`	Template 5 (Connection Accepted)	`NOTIFICATION`	**Dispatch ONLY IF sender's** `emailNotifications === true`
Weekly / Pending Digest	Background Cron Job	Activity Reminder Digest	`NOTIFICATION`	**Dispatch ONLY IF user's** `emailNotifications === true`
Email Registration OTP	`POST /register`	Template 1 (Registration OTP)	`TRANSACTIONAL`	**ALWAYS SEND** (Bypasses preference check)
Password Reset OTP	`POST /forgot-password/send-otp`	Template 2 (Password Reset OTP)	`TRANSACTIONAL`	**ALWAYS SEND** (Bypasses preference check)
Subscription Payment Receipt	`POST /razorpay/verify`	Template 6 (Subscription Confirmation)	`TRANSACTIONAL`	**ALWAYS SEND** (Bypasses preference check)

4. Frontend Preference Toggle Sync

When the user toggles **Email notifications** in the mobile app (`Profile` → `Preferences`), the mobile client issues:

```
PUT /notification/preferences
Content-Type: application/json

{
  "emailNotifications": false
}
```

The backend must persist this boolean in the user's `NotificationPreferences` record so the mailer immediately respects the user's choice.